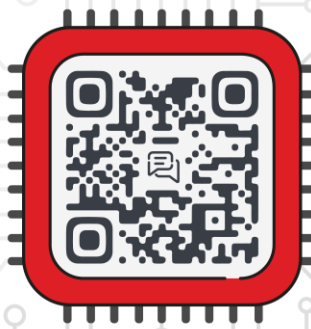




عملي

12

1440 sp

كلية الهندسة المعلوماتية

السنة الثانية

## ملحق تمارين داعمة للامتحان

د. عمل طلابي



برمجة ٣

**RB** Informatics; 9/01/2025

١. ما هو خرج الكود التالي :

```
class Parent{
public Parent(){
System.out.println("P");}}
class Child extends Parent{
public Child(){
System.out.println("C");}}
public class Test{
public static void main(String[]args){
Child c= new Child();
}}
```

الخرج: P

C

لان عند انشاء كائن من الصف الابن, يتم استدعاء  
باني الاب اولاً بشكل ضمني ثم باني الابن

٢. ما هو الخرج ؟

```
public class Main{
public static int calculate(){
try{
System.out.print("A");
return 1;}
catch(Exception e){
System.out.print("B");
return 2;}
finally{
System.out.print("C"); }}
public static void main(String[]args){
System.out.print(calculate());
}}
```

الخرج : AC1

اولاً لا يوجد أي كود يسبب استثناء لهذا الكود لذلك  
لن تنفذ كتلة catch و دائماً تعطى finally الاولوية  
للتنفيذ قبل ان يخرج التابع أي تنفذ قبل ال return  
اذن يُطبع حرف A ثم يحجز البرنامج قيمة  
1 و يطبع c ثم يطبع 1 .

ملاحظة هامة: ان وضعنا return3 مثلاً في بلوك ال

finally كانت ستتغير النتيجة و تلغى ال return الأولى .

٣. ما قيمة المتغير size بعد تنفيذ الكود ؟

```
import java.util.HashSet;
import java.util.Set;
public class CollectionTest{
    public static void main(String[]args){
        Set<String> mySet= new HashSet<>();
        mySet.add("Java");
        mySet.add("Python");
        mySet.add("Java");
        mySet.add("C++");
        int size = mySet.size();
        System.out.println(size);}}
```

الخرج: 3 : لان ال HashSet لا يسمح بالتكرار .

٤. أي سطر في الكود التالي سيسبب خطأ في الترجمة؟

```
1 abstract class Vehicle{
2     public abstract void startEngine();
3     public void stopEngine(){
4         System.out.println("Engine stopped.");
5     }
6 }
7 public class Car extends Vehicle{
8     //Missing implementation of startEngine()
9 }
```

الحل: Line 7 لان الكلاس Car يرث من كلاس مجرد Vehicle ولكنه لا يقوم بتطبيق override أي انه كلاس غير مكتمل

٥. ما هو الخرج ؟

```
class Animal{
    void makeSound(){
        System.out.println("Animal sound"); }
    class Dog extends Animal {
        @Override
        void makeSound(){
            System.out.println("Woof");}}
    public class Test{
        public static void main(String[]args){
            Animal myAnimal = new Dog();
            myAnimal.makeSound(); }}
```

الخرج : Woof



## ٦. ما هو الخرج ؟

```
public class StringTest{
    public static void main(String[]args){
        String s1= new String("hello");
        String s2="hello";
        String s3=s1.intern();
        System.out.print(s1==s2);
        System.out.print(",");
        System.out.print(s2==s3);}}
```

الحل: false,true

الشرح: new String("hello") هنا انت تشكل كائن جديد في الذاكرة Heap حتى لو كانت الكلمة موجودة سابقا اما s2="hello" هي نص ثابت ويتم تخزينه بمنطقة خاصة اسمها String Pool

اذا بما ان s1 و s2 كلاهما في اماكن مختلفة فهما لا

يتساويان , اما بالنسبة ل s2==s3 فان التابع intern وظيفته أن يبحث بال String Pool عن نص يطابق محتوى الكائن , ولان hello موجودة اصلا بـ String Pool فان s3 سوف تأخذ نفس المرجع و بالتالي المقارنة صحيحة

## ٧. ما هو الخرج ؟

```
class Outer{
    private int x=10;
    class Inner{
        public void display(){
            System.out.println("Value of x is"+x);}}}
    public class Main{
        public static void main(String[]args){
            Outer outer= new Outer();
            Outer.Inner inner = outer.new Inner();
            inner.display();}}
```

الخرج: Value of x is 10

## ٨. ما هو الخرج ؟

```
public class ExceptionTest{
    public static void main(String[]args){
        try{
            int [] arr = new int[6];
            System.out.println(arr[6]);
        }catch(ArithmeticException e){
            System.out.println("Arithmetic Error");}
        catch(Exception e){
            System.out.println("Generic Error"); }
        System.out.println("End of Program");}}
```

الخرج: سيتم طباعة "Generic Error" ثم

"End of program" لان محاولة الوصول ل arr[6]

ستسبب ArrayOutOfBoundsException ولانه لا

يوجد كتلة catch مخصصة لهذا الاستثناء ف سيتم

تنفيذ الاستثناء الاعم

١. نظام إدارة الكتب الصوتية (AudioBook System)

لدينا الواجهة MediaPlayer لديها الطرائق (Methods):

play(): تابع للتشغيل (حلقة تدور حسب مدة الكتاب وهي 60 دقيقة).

saveToHistory(): يضيف اسم الكتاب إلى سجل الاستماع وهو ملف نصي باسم History.txt

ولدينا الصف Book يطبق الواجهة MediaPlayer ويحتوي (اسم الكتاب، المؤلف، السعر).

المطلوب: عرّف الواجهة MediaPlayer والصف Book بالشكل المناسب، ثم اكتب البرنامج العام بحيث يقوم بما يلي:

قم بإنشاء مجموعة (Collection) من نوع ArrayList باسم myLibrary وقم بإضافة 3 كتب إليها يقوم المستخدم بإدخال بياناتها، بحيث يعتبر إدخال "سعر سالب" استثناءً يجب معالجته (Exception Handling).

قم بإضافة جميع أسماء كتب "الروايات" (إذا افترضنا وجود حقل للنوع) أو ببساطة أول كتابين من المجموعة إلى ملف History.txt باستخدام الطريقة saveToHistory.

أنشئ كتابين، ثم قم بإنشاء مهمتين من نوع PlayTask لتشغيل الكتابين (كل مهمة لتشغيل كتاب واحد)، ثم قم بتنفيذ المهمتين بحيث تنتظر المهمة الثانية انتهاء المهمة الأولى تماماً قبل أن تبدأ، وذلك بعد أن تعرف الصف PlayTask والذي يمكن تنفيذه كمهمة مستقلة (Thread).

الحل :

```
import java.io.*;
import java.util.*;
interface MediaPlayer {
    void play();
    void saveToHistory(); }
class Book implements MediaPlayer {
    String title;
    String author;
    double price;
    public Book(String title, String author, double price) {
        this.title = title;
        this.author = author;
        this.price = price; }
    @Override
    public void play() {
        for (int i = 1; i <= 60; i++) {
            System.out.println("Playing " + title + ": Minute " + i); }
    @Override
    public void saveToHistory() {
        try (FileWriter fw = new FileWriter("History.txt", true)) {
            fw.write(title + "\n");
        } catch (IOException e) {
            System.out.println("Error saving to file."); } } }
```

```
// Thread صف المهمة 3.
class PlayTask extends
Thread {
    Book book;
    public PlayTask(Book
book) {
        this.book = book;
    }
    public void run() {
        book.play();
    }
}
```

// البرنامج العام 4.

```
public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        ArrayList<Book> myLibrary = new ArrayList<>();

        // الطلب الأول: إضافة 3 كتب مع معالجة الاستثناء
        for (int i = 0; i < 3; i++) {
            try {
                System.out.println("Enter Title, Author, Price:");
                String t = sc.nextLine();
                String a = sc.nextLine();
                double p = sc.nextDouble();
                if (p < 0) throw new Exception("Price cannot be negative!");
                myLibrary.add(new Book(t, a, p));
            } catch (Exception e) {
                System.out.println(e.getMessage());
                i--; // لإعادة المحاولة
            }
        }

        // الطلب الثاني: إضافة الأسماء للملف
        for (Book b : myLibrary) {
            b.saveToHistory();
        }

        // مع انتظار الأولى Threads: الطلب الثالث
        Book b1 = new Book("Java Basics", "Author A", 100);
        Book b2 = new Book("Advanced Java", "Author B", 200);
        PlayTask task1 = new PlayTask(b1);
        PlayTask task2 = new PlayTask(b2);
        task1.start();
        try {
            task1.join(); // انتظار المهمة الأولى حتى تنتهي
        } catch (InterruptedException e) {
            e.printStackTrace();
        }
        task2.start();
    }
}
```

٢. نظام إدارة الموظفين (Employee Management System)

لدينا الواجهة Workable لديها الطرائق التالية:

performTask(): (حلقة تدور لمحاكاة إنجاز مهمة لمدة 8 ساعات).

saveReport(): تخزين اسم الموظف وساعاته في ملف نصي باسم Reports.txt.

ولدينا الصف Employee يطبق الواجهة Workable ويحتوي على (الاسم، القسم، الراتب).

المطلوب: عرّف الواجهة والصف بشكل مناسب، ثم اكتب البرنامج العام للقيام بما يلي:

أنشئ مجموعة من نوع Employee <HashMap<Integer, Employee> باسم staff. حيث يمثل المفتاح (Key) رقم الهوية (ID) ويمثل العنصر (Value) غرض الموظف.

قم بإضافة موظفين اثنين للـ Map، بحيث يقوم المستخدم بإدخال رقم الهوية وبيانات الموظف. عالج استثناءً في حال قام المستخدم بإدخال "راتب أقل من الحد الأدنى (مثلاً 1000)"، بحيث لا يتم إضافة الموظف وتظهر رسالة خطأ.

قم بالمرور على الـ Map، واستدع الطريقة saveReport لكل موظف موجود في القسم الذي يحدده المستخدم (مثلاً قسم "IT").

قم بإنشاء مهمتين (Threads) باسم ProjectTask لتنفيذ performTask لموظفين مختلفين، بحيث يتم تنفيذهما معاً في نفس الوقت (توازي تفرعي) دون انتظار :

```
import java.io.*;
import java.util.*;
interface Workable {
    void performTask();
    void saveReport(); }
class Employee implements Workable {
    String name;
    String department;
    double salary;
    public Employee(String name, String department, double salary) {
        this.name = name;
        this.department = department;
        this.salary = salary; }
    @Override
    public void performTask() {
        for (int h = 1; h <= 8; h++) {
            System.out.println(name + " is working: Hour " + h); } }
    @Override
    public void saveReport() {
        try (FileWriter fw = new FileWriter("Reports.txt", true)) {
            fw.write("Employee: " + name + " | Dept: " + department + "\n");
        } catch (IOException e) {
            System.out.println("File Error"); } } }
```

الحل

```
class ProjectTask extends Thread {
    Employee emp;
    public ProjectTask(Employee emp) { this.emp = emp; }
    public void run() {
        emp.performTask();
    }
}

public class CompanyApp {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        HashMap<Integer, Employee> staff = new HashMap<>();
        for (int i = 0; i < 2; i++) {
            try {
                System.out.println("Enter ID, Name, Dept, Salary:");
                int id = sc.nextInt();
                String n = sc.next();
                String d = sc.next();
                double s = sc.nextDouble();
                if (s < 1000) throw new Exception("Salary too low!");
                staff.put(id, new Employee(n, d, s));
            } catch (Exception e) {
                System.out.println(e.getMessage());
                i--;
            }
        }
        System.out.println("Enter department to save its reports:");
        String targetDept = sc.next();

        for (Employee e : staff.values()) {
            if (e.department.equalsIgnoreCase(targetDept)) {
                e.saveReport();
            }
        }
        Employee e1 = new Employee("Ahmad", "IT", 5000);
        Employee e2 = new Employee("Sara", "HR", 4500);
        ProjectTask t1 = new ProjectTask(e1);
        ProjectTask t2 = new ProjectTask(e2);
        t1.start();
        t2.start();
    }
}
```

٣. نظام إدارة الرحلات الجوية (Flight Management System)

لدينا الواجهة Flyable ولديها الطريقة:

takeOff(): (حلقة تدور من 1 إلى 10 لتمثيل مراحل الإقلاع).

لدينا الصف الأساسي Airline ويحتوي على:

الحقول: (اسم الشركة، رمز الطيران).

صف داخلي ساكن (Static Inner Class) باسم Flight:

يحتوي على: (رقم الرحلة، عدد الركاب).

يطبق الواجهة Flyable.

يحتوي على طريقة saveFlightInfo(): تخزن معلومات الرحلة في ملف Flights.txt.

المطلوب: اكتب البرنامج العام للقيام بما يلي:

أنشئ مجموعة من نوع Vector<Airline.Flight> باسم flightVector.

قم بإضافة رحلتين يدخل المستخدم بياناتهما، مع معالجة استثناء في حال كان "عدد الركاب" أكبر من سعة الطائرة القصوى (افترضها 300).

قم بالمرور على الـ Vector باستخدام Enumeration واستدع saveFlightInfo لكل رحلة.

قم بتعريف صف مهمة FlightThread ينفذ takeOff لرحلة معينة.

شغل مهمتين لرحلتين مختلفتين بحيث تنتظر الثانية انتهاء الأولى تماماً باستخدام join

الحل:

```
import java.io.*;
import java.util.*;
interface Flyable {
    void takeOff(); }
class Airline {
    static class Flight implements Flyable {
        String flightNum;
        int passengers;
        public Flight(String flightNum, int passengers) {
            this.flightNum = flightNum;
            this.passengers = passengers; }
        @Override
        public void takeOff() {
            for (int i = 1; i <= 10; i++) {
                System.out.println("Flight " + flightNum + " taking off: Step " + i); } }
        public void saveFlightInfo() {
            try (FileWriter fw = new FileWriter("Flights.txt", true)) {
                fw.write("Flight: " + flightNum + " | Passengers: " + passengers + "\n");
            } catch (IOException e) {
                System.out.println("File Error"); } } }
```

```
class FlightThread extends Thread {
    Airline.Flight flight;
    public FlightThread(Airline.Flight f) { this.flight = f; }
    public void run() {
        flight.takeOff();
    }
}

public class AirportApp {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        Vector<Airline.Flight> flightVector = new Vector<>();
        for (int i = 0; i < 2; i++) {
            try {
                System.out.println("Enter Flight Number and Passenger Count:");
                String num = sc.next();
                int pCount = sc.nextInt();
                if (pCount > 300) throw new Exception("Over Capacity! Max 300.");

                flightVector.add(new Airline.Flight(num, pCount));
            } catch (Exception e) {
                System.out.println(e.getMessage());
                i--;
            }
        }
        Enumeration<Airline.Flight> en = flightVector.elements();
        while (en.hasMoreElements()) {
            en.nextElement().saveFlightInfo();
        }
        Airline.Flight f1 = new Airline.Flight("GA101", 150);
        Airline.Flight f2 = new Airline.Flight("GA202", 280);
        FlightThread t1 = new FlightThread(f1);
        FlightThread t2 = new FlightThread(f2);
        t1.start();
        try {
            t1.join();
        } catch (InterruptedException e) {}
        t2.start();
    }
}
```

E. نظام إدارة المخازن الذكي SmartHome

ليكن لدينا الواجهة Control والتي لديها الطرائق التالية: `operate()` و `saveLog()`  
 حيث ان `operate()` تابع للتشغيل (حلقة تدور من 1 الى 6 لتمثيل مراحل الاقلاع مع غفوة ثانية)  
`saveLog()` يضيف اسم الجهاز و حالته لملف نصي باسم `HomeLog.txt`  
 ولدنيا الصف SmartHome يحوي (العنوان) و بداخله صف داخلي Device يطبق الواجهة Control ويحوي  
 (اسم الجهاز, استهلاك الطاقة).  
 والمطلوب: عرف الصفوف بالشكل المناسب ثم اكتب البرنامج العام للقيام بما ياتي:  
 انشئ كوليكتشن myDevice من نوع HashMap مفتاحها (رقم تسلسلي) و قيمتها كائن من نوع Device  
 قم باضافة جهازين يدخل المستخدم بياناتهما مع معالجة استثناء مخصص `PowerLimitException` اذا كان  
 استهلاك الطاقة اكبر من 2000 واط  
 قم بتعريف صف المهمة DeviceTask الذي ينفذ `operate()` لجهاز معين  
 انشئ مهمتين لجهازين مختلفين ونفذهما بحيث تنتظر المهمة الثانية انتهاء الاولى تماما قبل ان تبدأ وذلك  
 باستخدام التابع `join()`

//الاستثناء

```
public class PowerLimitException extends Exception {
    public PowerLimitException(String msg) {
        super(msg); } }
```

//الواجهة

```
public interface Control {
    public void operate();
    public void saveLog(); }
```

```
public class SmartHomeSystem {
```

```
    public static void main(String[] args) throws InterruptedException, PowerLimitException{
```

```
        SmartHome home = new SmartHome();
```

```
        HashMap<Integer, SmartHome.Device> myDevice = new HashMap<>();
```

```
        Scanner sc = new Scanner(System.in);
```

```
        int count=1;
```

```
        while(count<=2){
```

```
            // ادخال و معالجة الاستثناء
```

```
            try{
```

```
                System.out.print("Enter device Name: ");
```

```
                String name = sc.next();
```

```
                System.out.print("Enter Power: ");
```

```
                int p = sc.nextInt();
```

```
                if(p>2000) throw new PowerLimitException("Too much power!!!!");
```

```
                // انشاء كائن من الصف الداخلي
```

```
                myDevice.put(count, home.new Device(name,p));
```

```
                count++; }
```

الحل

```
catch(PowerLimitException e){
    System.out.println(e.getMessage());
} }
try{
    DeviceTask t1 = new DeviceTask(myDevice.get(1));
    DeviceTask t2 = new DeviceTask(myDevice.get(2));
    t1.start();
    t1.join();
    t2.start();
} catch(InterruptedException e){
    System.out.println("Thread interrupted");
}
}
```

// كلاس المهام

```
public class DeviceTask extends Thread{
    SmartHome.Device d;
    DeviceTask(SmartHome.Device d){
        this.d=d;
    }
    @Override
    public void run(){
        d.operate();
        d.saveLog();
    }
}
```

// الصف الخارجي و الداخلي

```
public class SmartHome {
    String address;
    public class Device implements Control{
        String deviceName;
        int power;
        public Device(String deviceName, int power) {
            this.deviceName = deviceName;
            this.power = power;
        }
        @Override
        public void operate() {
            try{
                for(int i=1;i<=6;i++){
                    Thread.sleep(1000);
                    System.out.println(deviceName+" running...");
                }
            } catch(InterruptedException e){
                System.out.println("Operation interrupted");
            }
        }
    }
}
```

```
@Override
```

```
public void saveLog() {
```

```
    try(FileWriter fw= new FileWriter("HomeLog.txt",true)){
```

```
        fw.write("Device: "+deviceName+" +Power: " + power +"\n");
```

```
    } catch (IOException ex) {
```

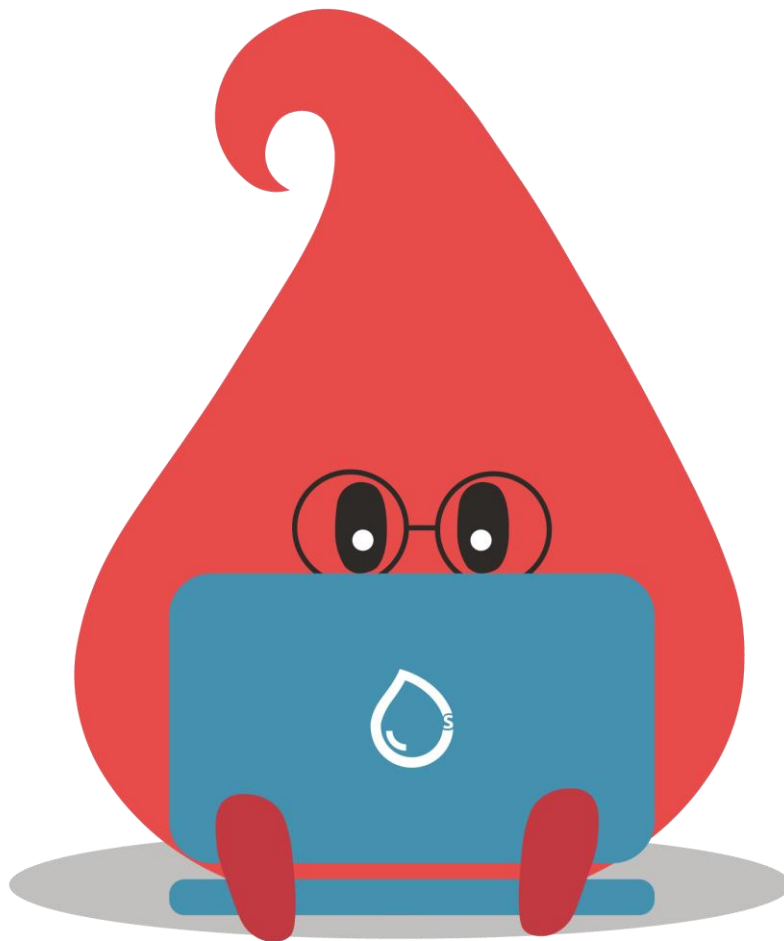
```
        System.out.println("Error writing log");
```

```
    }
```

```
}
```

```
}
```

```
}
```



"With this, we reach the end of the supplementary exercises.

Whishing you success and enjoyable coding".